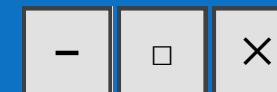


Monster Game.exe



PROJET BTS SIO — OPTION SLAM

Monster Game

Jeu web de type Tamagotchi — application full-stack

Simon BOURMAUD & Matthieu CAZALIS

YNOV Campus Bordeaux • Avril – Juin 2026

Épreuve U6 — Conception et développement d'applications

Un binôme de développeurs

SB

Simon Bourmaud

Développeur full-stack

- Frontend React + backend Express
- Début de base de donnée
- Tutoriel contextuel & popup de complétion

MC

Matthieu Cazalis

Développeur full-stack

- Frontend React + backend Express
- Style visuel Windows 98 / rétro
- Gestion des pages & monnaie

Travail en binôme

Développement en parallèle sur des branches Git séparées, puis fusion régulière du code.

Un Tamagotchi web

Le besoin

- Concevoir un jeu web complet, de la base de données à l'interface
- Mettre en pratique le full-stack vu en formation
- Travailler à plusieurs avec Git

Le concept

- Le joueur adopte un monstre virtuel
- Il le fait évoluer à intervalles réguliers
- Il gagne des pièces, puis achète espèces & décorations

L'idée forte

Le temps comme mécanique de jeu

- Chaque évolution déclenche un compte à rebours (cooldown).
- Ce timer est stocké en base et vérifié côté serveur :
- → **impossible de tricher en modifiant le navigateur.**

Style visuel : thème Windows 98

Architecture 3 couches & stack utilisée

FRONTEND

Couche présentation

React 18 · TypeScript · Vite · React Router · CSS Windows 98

HTTP / JSON

BACKEND

Couche logique / métier

Node.js · Express · TypeScript · API REST · JWT · bcrypt

SQL

BASE DE DONNÉES

Couche données

MySQL 8 · Docker · phpMyAdmin · 6 tables relationnelles

Fonctionnalités marquantes

1 Authentification sécurisée

Inscription / connexion par JWT, mots de passe hashés avec bcrypt.

2 Évolution temporisée

Timer d'évolution stocké en base et contrôlé côté serveur.

3 Boutique & économie

Achat d'espèces et de décorations avec une monnaie de jeu (coins).

4 Inventaire & collection

Monstres en cours et terminés, histoire du monstre débloquée à la complétion.

5 Tutoriel contextuel

Spotlight qui met en surbrillance les éléments réels de l'interface.

6 Personnalisation

Étagère de décorations + thème de couleurs configurable.

Qui a fait quoi ?

Simon Bourmaud

- Système d'authentification (JWT, bcrypt)
- Database
- Tutoriel contextuel (spotlight) & popup de complétion
- Boutique & achats

Matthieu Cazalis

- Identité visuelle Windows 98
- Préparation des pages & composantes
- Logique d'évolution & timer côté serveur
- Intégration des assets graphiques

En commun

Conception du modèle de données • Architecture de l'API • Fusion des branches Git & résolution des conflits • Tests et débogage

Credits supplémentaires

Artiste : Mad.davvg101

Références : Pinterest & SpaceHey

Nos réussites



Timer anti-triche

Le contrôle du cooldown côté serveur : une vraie logique métier sécurisée, pas juste de l'affichage.



Tutoriel contextuel & customisation

Un spotlight dynamique qui se positionne sur les vrais éléments via `getBoundingClientRect()`. Et possibilité de changer des couleurs avec `LocalStorage`.



Cohérence visuelle

Un thème Windows 98 tenu sur toute l'application, jusqu'aux popups et boutons interactifs. Monstres dessinés sur-mesure pour le projet.



Travail collaboratif

Deux versions développées en parallèle puis fusionnées proprement avec Git.

Obstacles & solutions

Fusion des branches Git

Problème : Nos deux versions divergeaient beaucoup.

Solution : Fusion méthodique, résolution des conflits fichier par fichier, harmonisation du style.

Configuration CORS

Problème : Le front (5173) n'arrivait pas à appeler le back (3000).

Solution : Mise en place du middleware CORS côté Express pour autoriser l'origine du frontend.

Positionnement du tutoriel

Problème : L'info-bulle se plaçait mal selon l'élément ciblé.

Solution : Calcul dynamique avec `getBoundingClientRect()` + repositionnement selon l'espace dispo.

Synchronisation des coins

Problème : Le solde ne se mettait pas à jour après un levelup.

Solution : Événement JavaScript personnalisé écouté par la Navbar pour rafraîchir sans recharger.

Ce que le projet nous a appris

Sur le plan technique

- Construire une application full-stack de bout en bout
- Sécuriser une API (JWT, bcrypt, contrôle serveur)
- Concevoir et requêter une base relationnelle
- Conteneuriser un service avec Docker
- Découper le code en composants réutilisables

Sur le plan humain

- Travailler à plusieurs sur une même base de code
- Utiliser Git en équipe (branches, fusions, conflits)
- Communiquer pour répartir et coordonner les tâches
- Faire des compromis pour harmoniser deux visions
- Gérer un projet dans la durée et tenir les délais

Ce que nous aimerions ajouter

+ Notifications

Prévenir le joueur quand son monstre est prêt à évoluer (push / email).

+ Dimension sociale

Classement entre joueurs, visite de l'espace des amis.

+ Plus de contenu

Nouvelles espèces, décorations et événements saisonniers.

+ Mini-jeux

Gagner des coins autrement que par le timer.

+ Déploiement en ligne

Héberger le jeu pour le rendre accessible publiquement.

CONCLUSION

Un projet full-stack complet, mené en équipe

Monster Game nous a permis de couvrir tout le cycle de développement : conception de la base, API sécurisée, interface soignée, et collaboration Git.

Merci de votre attention ! Avez-vous des questions ?

Simon BOURMAUD & Matthieu CAZALIS • BTS SIO SLAM •
Session 2026